

POP Temat 5 Dokumentacja końcowa

Brygida Silawko

Tomasz Smoleń

7 października 2026

1 Opis problemu

Problem dotyczy zredukowania zjawiska zbyt dużej zbieżności w algorytmach ewolucyjnych co skutkuje w utykaniu w optimach lokalnych. Algorytmy ewolucyjne mają różne mechanizmy, które odpowiadają za eksplorację (np. mutacja) oraz eksploatację (selekcja, krzyżowanie), koniec końców często algorytm będzie albo za bardzo eksploatował (przez co będzie łatwo utykał w optimach lokalnych), albo za bardzo eksplorował (przez co czas jego działania będzie bardzo długi). Rozwiązaniem tego problemu jest dodanie heurystyki, która steruje aktualnym zachowaniem algorytmu. My skupiamy się na heurystyce DG. Heurystyka DG (Diversity Guided) stosuje mechanizm dwóch faz, Eksploracji, podczas której następują mutacje populacji, oraz Eksploatacji, podczas której następuje krzyżowanie oraz selekcja. Do sterowania, w której fazie się znajdujemy stosujemy wyznacznik ‘diversity’, który wyliczamy następująco:

$$diversity(P) = \frac{1}{|L| \cdot |P|} \cdot \sum_{i=1}^{|P|} \sqrt{\sum_{j=1}^N (s_{ij} - \bar{s}_j)^2} \quad (1)$$

$|L|$ - długość przekątnej w przestrzeni przeszukiwań $S \subseteq \mathbb{R}^N$

P - populacja, $|P|$ - rozmiar populacji

N - wymiarowość problemu

s_{ij} - j -ty element i -tego osobnika

$\bar{s}_j$ - j -ta wartość średniego punktu $\bar{s}$

W równaniu (1) normalizujemy wynik sumy po i , poprzez dzielenie długość przekątnej przestrzeni przeszukiwań (tym większa wymiarowość problemu tym większa suma po j)

W zależności od tego czy nasze ‘diversity’ jest większe lub mniejsze od wartości d_{min} i d_{high} zmieniamy, w której fazie się znajdujemy. Dzięki temu algorytm eksploruje, kiedy różnorodność naszej populacji będzie maleć, a eksploatuje, kiedy różnorodność jest zbyt duża.

2 Wykonane eksperymenty numeryczne

Przeprowadziliśmy dwa główne eksperymenty:

2.1 Krzywe ECDF na podstawie benchmarku BBOB

Skorzystaliśmy z benchmarku BBOB z frameworku coco do zbadania i porównania naszego algorytmu z implementacją ewolucji różnicowej w wariancie rand/1/bin z biblioteki scipy. Benchmark uruchomiliśmy dla wszystkich funkcji na wymiarach 2, 3, 5, 10, 20 i 40.

2.2 Zbadanie zbieżności na funkcji sferycznej

Znowu wspomogliśmy się frameworkiem coco do porównania zbieżności naszego algorytmu oraz implementacji DE/rand/1/bin z scipy. Ten eksperyment wykonaliśmy dla funkcji sferycznej (funkcja f1 z coco) dla wymiaru 5D.

3 Wybór technologii

Projekt został zrealizowany w języku Python, przy wykorzystaniu bibliotek takich jak numpy. Dodatkowo do eksperymentów będziemy wykorzystywać biblioteki coco-experiment i cocopp. Jako menadżera paczek użyliśmy uv.

4 Pseudokod zmodyfikowanego DE/rand/1/bin

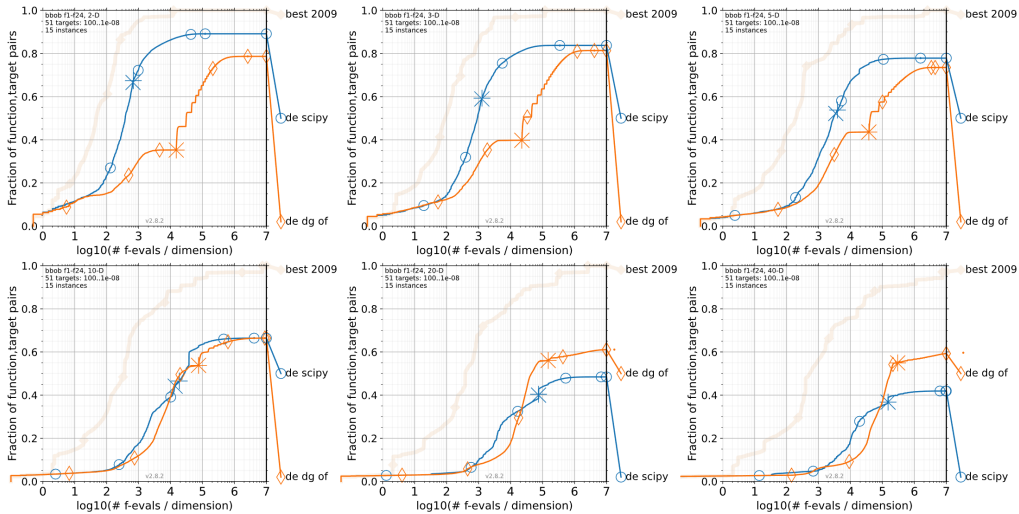
W porównaniu z pseudokodem z dokumentacji wstępnej zmienił się sposób wybierania punktów do krzyżowania i mutacji oraz ich oceny. Wcześniej w krzyżowaniu i mutacji brał udział punkt p_i , teraz wykorzystywany jest jedynie w selekcji, gdzie porównujemy go z nowym osobnikiem powstałym w wyniku krzyżowania lub mutacji. Ten sposób jest bliższy klasycznemu DE, gdzie i -ty punkt z populacji (p_i) jest porównywany z nowym osobnikiem stworzonym z innych wylosowanych punktów roboczych. Dzięki wprowadzeniu tych zmian nasz algorytm miał większą liczbę sukcesów w benchmarku bbof.

5 Wyniki eksperymentów

5.1 Benchmark BBOB

Wyniki benchmarku BBOB prezentują się następująco:

- de scipy - implementacja DE z scipy
- de dg of - nasz algorytm



Rysunek 1: wyniki benchmarku BBOB

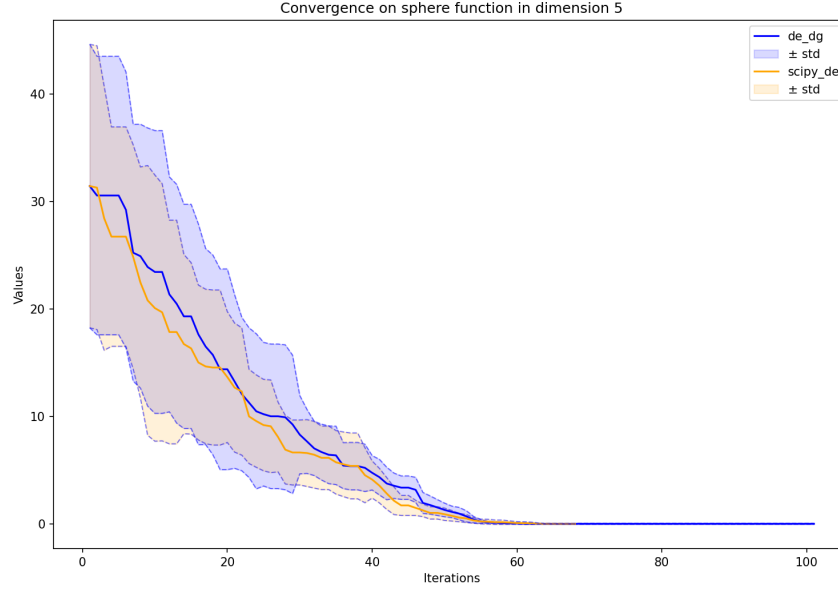
Widzimy, że generalnie nasz algorytm osiąga gorsze wyniki dla niższych wymiarowości i wychodzi na prowadzenie przy wyższych, jednak więcej na ten temat będzie w sekcji Analiza uzyskanych wyników

Algorithm 1 DE/rand/1/bin z DG

Inicjuj populację początkową P_0 o rozmiarze μ
 $t \leftarrow 0$
mode $\leftarrow$ Exploit
while !stop **do**
 $P_r \leftarrow P_t$ $\triangleright$ Populacja robocza reprezentująca stan populacji P_t po zmianach osobników od 1 do i
 for $i \in 1 : \mu$ **do**
 if $diversity(P_r) < d_{low}$ **then**
 mode $\leftarrow$ Explore
 else if $diversity(P_r) > d_{high}$ **then**
 mode $\leftarrow$ Exploit
 end if
 $p_i = P_t[i]$
 if mode == Exploit **then**
 $c_1, c_2 \leftarrow sample(P_t \setminus P_t[i], 2)$
 for $j \in 1 : n$ **do** $\triangleright n$ - długość osobnika
 if $rand(0, 1) < p_c$ **then** $\triangleright p_c$ - p-ństwo krzyżowania
 $z_{i,j} \leftarrow c_{1,j}$
 else
 $z_{i,j} \leftarrow c_{2,j}$
 end if
 end for
 if $ocena(z_i) > ocena(p_i)$ **then** $\triangleright$ lub na odwrót w przypadku minimalizacji
 $P_r[i] \leftarrow z_i$ $\triangleright$ Podmień i-tego osobnika na wynik krzyżowania
 end if
 else $\triangleright$ mode == Explore
 $x_1, x_2, x_3 \leftarrow sample(P_t \setminus P_t[i], 3)$
 $m_i \leftarrow x_3 + F(x_1, x_2)$
 if $ocena(m_i) > ocena(p_i)$ **then** $\triangleright$ lub na odwrót w przypadku minimalizacji
 $P_r[i] \leftarrow m_i$ $\triangleright$ Podmień i-tego osobnika na wynik mutacji
 end if
 end if
 end for
 $P_{t+1} \leftarrow P_r$
 $t \leftarrow t + 1$
end while
Wybierz najlepszego osobnika z P_t

5.2 Badanie zbieżności

Widzimy że oba algorytmy zbiegają bardzo podobnie a różnice pomiędzy nimi są pomijalne.



Rysunek 2: Wykres zbieżności, Values - wartości funkcji celu

6 Analiza uzyskanych wyników

W celu analizy uzyskanych wyników wykonaliśmy test Wilcoxona dla par obserwacji, żeby sprawdzić, czy różnice pomiędzy algorytmami są statystycznie znaczące. Weryfikowaliśmy hipotezę zerową mówiącą o tym, że rozkłady różnic wyników obu algorytmów są symetryczne wokół zera (czyli algorytmy działają tak samo). Hipoteza alternatywna zakładała, że istnieje jakakolwiek różnica między nimi, bez zakładania z góry, który jest lepszy. Dla każdego wymiaru został wykonany jeden test. Parą w teście była mediana wartości Δf z 15 uruchomień danej funkcji dla algorytmu z scipy i mediana wartości Δf z 15 uruchomień danej funkcji dla naszego algorytmu, gdzie:

- $\Delta f = f(x_{best}) - f(x_{opt})$
- x_{best} - najlepsze rozwiązanie znalezione przez algorytm
- x_{opt} - rozwiązanie optymalne

Dodatkowo zastosowaliśmy próg odcięcia na poziomie 10^{-8} , wszystkie Δf poniżej tej wartości są traktowane jak 10^{-8} . Zrobiliśmy tak, ponieważ w BBOB wartość błędu poniżej 10^{-8} są uznawane za sukces.

W tabeli wyników:

- Scipy_Median i DE_DG_Median to mediana z median Δf
- Wins - ilość wygranych algorytmu z scipy $\Delta f_{scipy} < \Delta f_{de_dg}$
- Ties - $\Delta f_{scipy} = \Delta f_{de_dg}$
- Losses - ilość przegranych algorytmu z scipy $\Delta f_{scipy} > \Delta f_{de_dg}$

Dimension	Num_Functions	Scipy_Median	DE_DG_Median	Wins	Ties	Losses	P_Value
2	24	8.30e-07	1.80e+00	20	1	3	0.0001
3	24	1.29e-05	8.15e-01	17	2	5	0.0003
5	24	3.59e-05	5.60e-01	14	3	7	0.0064
10	24	9.60e-04	1.00e-03	9	3	12	0.3392
20	24	9.15e-01	1.25e-01	8	2	14	0.9095
40	24	1.95e+00	1.25e-01	8	2	14	0.9353

Tabela 1: Wyniki testu Wilcozona

Dla wymiarowości 2, 3 i 5 wartość p-value < 0.05 , co pozwala odrzucić hipotezę zerową. Różnica między algorytmami jest istotna statystycznie. Analizując liczbę wygranych, możemy stwierdzić, że standardowa implementacja DE/rand/1/bin z biblioteki SciPy radzi sobie lepiej dla niskich wymiarowości.

W przypadku wyższych wymiarów uzyskane p-value jest znacznie powyżej progu istotności $\alpha = 0.05$. Oznacza to, że nie ma podstaw do odrzucenia hipotezy zerowej. Statystycznie oba algorytmy wykazują porównywalną skuteczność. Mimo że DE/rand/1/bin z biblioteki SciPy, częściej przegrywał niż wygrywał z naszym algorytmem, nie możemy stwierdzić, że był on gorszy.

7 Dyskusja wyników

Niestety, heurystyka DG nie sprostała oczekiwaniom i odnosi generalnie gorsze od typowego DE wyniki. Nie rozwiązała problemu zbyt szybkiej zbieżności algorytmów, a dorzuciła dodatkowe obliczenia podczas kalkulowania diversity, które mogą być bardzo drogie, przez co czas działania naszego algorytmu w benchmarku BBOB był znacznie dłuższy od implementacji DE z scipy.